

ScamDunk — Full Codebase Audit

Repository: `Elimiz21/scam-dunk-re-write-claude-code` · Date: 2026-06-11 · Branch reviewed: `main`
(HEAD 30976b1)

Scope: every production source file — Next.js app (68,213 LOC TS/TSX), Python AI service (~9,600 LOC), evaluation pipeline (25,669 LOC), Prisma schema, 9 GitHub workflows, deployment configs.

Method: 7 parallel adversarial line-by-line reviews (AI scoring, Python ML, social scanning, security, architecture/data, admin UI ×2, public frontend) + automated toolchain checks, cross-verified.

21

CRITICAL

65

HIGH

115

MEDIUM

71

LOW

272

TOTAL FINDINGS

Contents

1. Executive summary

2. Act today — emergency items

3. Why the AI scoring isn't working

4. Why the social scans are unstable & expensive (and the Apify verdict)

5. Security review

6. Architecture & data layer

7. Frontend & admin UI

8. Automated check results

9. Findings index by area

10. What is working correctly

11. Remediation roadmap (summary)

12. Methodology & appendices

1. Executive summary

ScamDunk is a real, substantial product — a Next.js 14 scan app with PayPal/Apple billing, a 26-page admin intelligence dashboard, a Python ML sidecar on Railway, and a nightly GitHub Actions pipeline scoring ~7,000 stocks. The audit found the product's *core promise* — accurate AI risk scoring — undermined by a set of specific, fixable engineering defects rather than by any single design mistake. Three headline conclusions:

1. **The "AI" layer is largely not running.** The Random Forest model never produces a prediction in production (a 49-vs-35 feature mismatch makes it throw, retrain on synthetic data, and return 0.0 — on every request). The LSTM was trained on synthetic absolute prices and emits noise on real data. The SEC check in the Python service is a hardcoded 8-ticker demo list. What users actually receive is the rule-based score — and that layer over-flags small caps so aggressively (HIGH≥5 with 4–5 correlated signals stacking on a single price event) that essentially every OTC stock reads HIGH, while silent data-gap handling simultaneously produces false LOWs. Meanwhile the evaluation harness scores with a *different engine* than production, so every calibration number measured to date describes a scorer that isn't the one users see.

2. **The social scans fail by construction, and Apify is not the cost.** Apify appears nowhere in this codebase — source, lockfile, workflows, or git history. If you're paying for it, it's orphaned spend: cancel it. The real per-scan cost is Serper + Perplexity + OpenAI screening ($\approx$ \$0.15–0.30/clean scan), and the real bill multiplier is instability: the scan runs synchronously inside a 300-second Vercel function whose internal budgets exceed 300s, results are written only at the very end (a killed function loses everything already paid for), and the pipeline's fallback then *re-spends* the same API budget locally. The Reddit scanner alone needs ≥ 125 s of sleep against a 120s timeout — it has mathematically never completed a full 50-ticker scan.
3. **One credential leak requires action today.** The production Supabase database password is committed in `scripts/query-blogs-pg.mjs` (and git history). Rotate it now. Beyond that, the security posture is mixed: webhook verification, token handling and admin-route auth are largely done well (65 admin routes audited, zero missing auth on mutations), but there are real holes — an unmetered twin of the paid scan endpoint, Apple receipt replay enabling unlimited PAID accounts from one purchase, no rate limit on the web login, and a preview-login backdoor that creates an OWNER admin with a published default password against the shared production DB.

A phased remediation roadmap accompanies this report (`ROADMAP.md`): Phase 0 emergency items (~2 hours), then six weeks of sequenced work. The two highest-leverage engineering moves are (a) unifying the scoring engine into one shared module and recalibrating it, and (b) making the social scan asynchronous with incremental writes. Together they resolve the two complaints this audit was commissioned for.

2. Act today — emergency items

E1 — Rotate the production database password (verified leak)

`scripts/query-blogs-pg.mjs:6` contains the full production connection string — host `aws-1-ap-northeast-1.pooler.supabase.com`, project ref and password — committed and in git history. Anyone with repo read access (collaborators, any leaked clone or fork) can connect directly to production Postgres: all user data, password hashes, billing IDs, support tickets, bypassing the app entirely.

Do now: Supabase dashboard → Settings → Database → reset password; update Vercel/GitHub Actions env vars; replace the literal with `process.env.DATABASE_URL`; audit DB logs for unfamiliar access. History purge (`git filter-repo`) when convenient — rotation is what closes the hole.

⚠ E2 — Remove the preview-login OWNER backdoor

`/api/admin/auth/preview-login` creates a persistent **OWNER**-role admin (`preview@scamdunk.com`) with default password `PreviewAdmin2026!` when `PREVIEW_ADMIN_PASSWORD` is unset. Preview deployments share the production database, so this account works against production `/admin/login`. Delete the row if present; require a strong env password; fail closed.

⚠ E3 — Cancel Apify; set the Python API secret

Apify is not integrated anywhere in this repository — it is orphaned spend (see §4). Separately, the Python AI service serves all endpoints *unauthenticated* if `AI_API_SECRET` is unset (it only logs a warning) — verify the secret is set on Railway and make the service refuse to start without it.

3. Why the AI scoring isn't working

Full detail: `findings/01-ai-scoring-typescript.md` (39 findings) and `findings/02-python-ai-service.md` (35 findings).

The scoring pipeline has three layers — TypeScript rule engine (`src/lib/scoring.ts`), Python ML service (`python_ai/`), and an LLM narrative — and each contributes a distinct failure mode. Ranked by impact:

3.1 The ML ensemble is functionally absent

CRITICAL `python_ai/feature_engineering.py:444–506` vs `ml_model.py:84–95`, `pipeline.py:730–751`

Random Forest probability is always 0.0 — and the model retrains on every request

Serving emits 49 features; the model was trained on 35, in a different order, with different casing (`Price_Change_3d` vs `price_change_3d`). `scaler.transform()` throws, the pipeline catches it, retrains a fresh RF on *synthetic* data (per request — seconds of CPU and a disk write), retries, fails again, and sets `rf_prob = 0.0`. Half of the advertised ensemble contributes nothing, ever, while consuming most of the latency budget.

CRITICAL `python_ai/lstm_model.py:101–107`, `393–397`, `499–505`

LSTM scores are noise on real data

The LSTM consumes raw `Close` / `Volume` scaled by a MinMaxScaler fit on synthetic ranges (\$1–\$300, 10k–5M shares). A real \$0.40 stock trading 80M shares — the product's core target — lands far outside [0,1] and saturates the network. With RF at 0.0, the "AI score" is in reality the rule-based anomaly heuristics plus hardcoded probability floors.

CRITICAL `python_ai/data_ingestion.py:417–421`, `463–465`; `config.py:194–196`

OTC stocks are misclassified as non-OTC; failed lookups fabricate random large-cap fundamentals

yfinance reports Pink Sheets/OTCQX/OTCQB as `PNK` / `OQX` / `OQB` — none match the OTC detection sets, so the weight-3 OTC signal, the more-sensitive OTC thresholds, and every OTC probability floor are dead *for exactly the stocks the product targets*. Worse: when the (heavily rate-limited) yfinance `.info` call fails, the code silently substitutes **random** fundamentals — market cap uniformly drawn from \$0.5–50B on a random major exchange — so a real penny-stock scam can be scored as a healthy mid-cap, differently on every restart.

3.2 The rule engine over-flags by construction

CRITICAL `src/lib/scoring.ts:553–555 + 256–457`

HIGH $\geq$ 5 / MEDIUM $\geq$ 2 with 4–5 correlated signals stacking on one event

A single price pump fires up to five signals at once (SPIKE_7D 3–4, SPIKE_3D 2–3, PRICE_ANOMALY 2–4, EXTREME_SURGE 3, PRICE_ACCELERATION 2); one volume event fires four more. The HIGH cutoff was lowered to 5 without de-duplicating them. Consequences: a NYSE stock up 25% on earnings with 3 $\times$ volume = HIGH; any sub-\$300M or sub-\$5 stock = at least MEDIUM; an OTC stock with a routine 10% weekly move = HIGH. Acceleration signals have no magnitude floor (three +0.1% days fire). Result: the false-positive flood users perceive as "scoring doesn't work".

CRITICAL `evaluation/scripts/standalone-scorer.ts vs src/lib/scoring.ts`

The evaluation harness scores with a different engine than production

The nightly pipeline's "layer1_deterministic" is `standalone-scorer.ts` — which gates patterns at 7 days of history (prod: 30), uses `Math.abs` where prod is signed, guards missing marketCap (prod doesn't), fixes the RSI flat-series bug (prod returns 100), and lacks the entire z-score anomaly stack. Every precision/recall number ever measured — including the test script that recommended lowering HIGH to 5 — describes a scorer that is not the one in production. **This is the root cause of "it looked fine in eval but doesn't work live."**

CRITICAL `src/lib/scoring.ts:200–221; src/lib/marketData.ts:157–161, 737–742`

Missing data manufactures both false HIGHS and false LOWs — silently

Missing price/marketCap fields default to `0`, which reads as "microcap + penny stock + no liquidity" = +6 $\rightarrow$ instant HIGH for any stock with a data gap (including mega-caps). In the opposite direction, when the FMP history endpoint 403s (it's plan-gated) or Alpha Vantage throttles, the quote still succeeds, `dataAvailable` stays true, and ALL pattern/anomaly detection silently turns off — an actively pumping stock scores LOW with no indication that 13 of 18 signal types never ran. Same ticker, wildly different scores depending on which upstream call failed that minute — the instability you've observed.

3.3 The regulatory layer is unreliable at both ends

CRITICAL `src/lib/marketData.ts:920–931; src/lib/regulatoryDatabase.ts:203–249; src/lib/scoring.ts:618–629`

SEC checks: substring-match false positives, a probably-dead live check, garbage-ticker ingestion — and suspended stocks come back "INSUFFICIENT"

`checkSECFeed` substring-matches the ticker against the entire EDGAR XML ("GE" matches "EXCHANGE"; weight 5 → forced HIGH), sends no User-Agent (EDGAR 403s such requests, so in production it likely always returns false — silently dead), and the suspension sync extracts "tickers" as any 2–5 letter capitalized word ("ORDER", "CORP" become CRITICAL-flagged tickers). The Python side's SEC integration is openly fake (`SEC_FLAGGED_TICKERS = { 'SCAM', 'PUMP', 'DUMP', ... }`). And because the alert check is gated on having a quote, SEC-suspended stocks — the highest-risk category in the product — typically return INSUFFICIENT instead of HIGH.

3.4 Integration defects mask all of the above

CRITICAL `src/app/api/check/route.ts:124–237, 541–567; ai-analyze/route.ts:99–148`

The 503 alert path is dead code; backend payloads are consumed unvalidated

`ServiceUnavailableError` is thrown *inside its own try/catch* and converted to `{success:false}` — the elaborate 503 handler and admin-alert email can never execute from this path. The TS client also aborts the Python call at 10s while the Python path does a per-request retrain + uncached yfinance fetches + (for HIGH results) news verification — so the AI answer, when it exists, frequently arrives too late and is discarded. Backend JSON is consumed without validation: a missing weight makes `totalScore = NaN`; an unknown risk level passes a lying cast; level and score come from different systems and can contradict. And `isLegitimate` is redefined on the AI path so an obscure ticker scored LOW-with-no-signals is presented with the "well-established company" reassuring narrative — the exact false-negative messaging a scam victim would screenshot.

HIGH `src/lib/scoring.ts:574–597; src/lib/narrative.ts:181–192`

Incoherent outputs erode trust

A stock can be simultaneously `riskLevel=HIGH` and `isLegitimate=true`, sending the LLM contradictory instructions ("RISK LEVEL: HIGH" + "tone should be reassuring"). Signal descriptions overstate evidence 2× (SPIKE_THEN_DROP says "50%/+40%+", triggers at 25%/20%). Z-score signals look only at the single most recent day, so ~5% of perfectly normal days randomly add +2 (= MEDIUM alone) and identical scans disagree day to day. Crypto is effectively unscored: every coin is labeled "OTC" (+3, BTC = permanent MEDIUM) while CoinGecko's 4-day candles silently disable all pattern detection — and three real NYSE tickers (SOL, APT, LINK) are scored against *cryptocurrency* data due to an override bug.

The fix, in order (Phase 2 of the roadmap)

1. Extract one shared pure scoring module used by app + pipeline; delete the fork; re-calibrate on the labeled set.
2. De-duplicate correlated signals (max-of-family), add magnitude floors, re-derive cutoffs.
3. Treat missing data as unknown (skip signal + confidence field), not zero.
4. Fix the RF feature contract + stop in-request retraining; fix OTC code mapping; remove the synthetic-fundamentals fallback; disable the LSTM until real training data exists.
5. Replace the SEC feed checks with structured data + proper User-Agent; force HIGH for suspended tickers regardless of quote availability.
6. Validate the backend contract with zod; fix the swallowed 503; raise the AI timeout to ~20s after removing the retrain.

4. Social scans: instability, cost, and the Apify verdict

Full detail: `findings/03-social-scan.md` (46 findings).

Apify: definitively not used

Zero references in source, `package.json`, lockfile, env examples, workflows, or git history (`git log -S apify`). The only hit is an unrelated marketing-skill reference doc. **If you are paying for an Apify subscription, nothing in this repository consumes it — cancel it.**

4.1 What actually runs, and what it costs

The nightly pipeline POSTs up to 50 HIGH-risk tickers to `/api/admin/social-scan` (Vercel, `maxDuration=300`), which synchronously runs six scanners in parallel — Serper (paid), Perplexity sonar (paid), Reddit public JSON, YouTube API, StockTwits, Discord bot — then screens flagged mentions with GPT-4o-mini (paid) and writes everything to the DB in one batch at the very end.

Cost line	Per clean 50-ticker scan	Notes
Serper	~200 credits ≈ \$0.06–0.20	2 queries/ticker × <code>num: 15</code> = 2 credits each — the largest line item
Perplexity sonar	≈ \$0.08	10 batched calls; 5–15s each — frequently lose the 120s race <i>after being paid for</i>
OpenAI screening	≈ \$0.01	gpt-4o-mini, batches of 25
YouTube / Reddit / StockTwits / Discord	\$0	but one scan = 50% of the free YouTube daily quota
Total (clean)		at one cron run per weekday

	$\approx \$0.15\text{--}0.30 \rightarrow \sim \$3\text{--}7/\text{month}$	
--	---	--

Why the real bill is higher: on timeout days (which are the norm, see below) the pipeline's local fallback *re-spends* Serper+Perplexity on the same 50 tickers while the deployed run may still be spending ($\times 2$); manual admin re-triggers have no concurrency guard; abandoned scanners keep making paid calls whose results are discarded; and the same post is screened and stored once per substring-matching ticker ($1.5\text{--}3\times$). If your actual spend is far above $\sim \$20/\text{month}$, the excess is in the OpenAI news-analysis phases of the nightly pipeline or in orphaned external subscriptions — compare provider dashboards against the table.

4.2 Why scans fail (ranked)

CRITICAL `src/lib/social-scan/scanners.ts:28,186,546; orchestrate.ts:289`

The Reddit scanner can never finish: 125s of mandatory sleep vs a 120s timeout

2.5s politeness delay $\times$ 50 tickers = 125 seconds of sleep alone against `SCANNER_TIMEOUT = 120_000`; StockTwits ($2s \times 50$) usually times out too. Because each scanner returns results only at the *end* of `scan()`, `Promise.race` discards 100% of partial results. Every full scan is PARTIAL by construction.

CRITICAL `src/app/api/admin/social-scan/route.ts:19,364; orchestrate.ts:359`

A 2–5-minute synchronous job inside a serverless request, with results written only at the end

Internal budgets (120s scanners + 180s AI screening + aggregation + write) exceed the 300s ceiling in the worst case; the pipeline client aborts at exactly 5 minutes. A killed function loses *everything already paid for* and leaves the run stuck RUNNING until a later GET marks it TIMED_OUT. Git history shows this exact failure was "fixed" twice before by shrinking timeouts — the architecture, not the numbers, is the problem.

HIGH `scanners.ts:25–73 (Reddit); all fetches`

Datacenter-IP blocking + no fetch timeouts + no real retry policy

Reddit blocks Vercel egress IPs (the code's own comments and HTML-block detection acknowledge it) — the scanner burns its 2-minute window for nothing nearly every serverless run. No fetch anywhere in the scanner file has an AbortController; one hung socket eats a scanner's entire budget. 429s get a single naive retry on Reddit only.

HIGH `prisma/schema.prisma:789–815; orchestrate.ts:473; ingest/route.ts:90–121`

No dedup, no idempotency

`SocialMention` has no unique constraint, so `skipDuplicates: true` is a no-op: the same post is re-inserted every daily run and once per ticker it substring-matches. The ingest route inserts row-by-row with no `maxDuration` (default $\sim 10s$) — partial ingests, and retries re-insert everything. Dashboard counts are inflated; the DB grows without bound.

HIGH evaluation/scripts/social-scan/browser-agents/*; src/app/api/admin/browser-agents/
route.ts:122–176

The browser-agent subsystem is dead code with live risks

It cannot run — `playwright`, `playwright-extra` and `otplib` are not dependencies anywhere. The admin "trigger scan" button queues PENDING sessions that *nothing processes* ("Browser scan queued" is a lie), its persistence paths resolve to a nonexistent directory, and its evidence collector saves an empty file. Yet it is wired to collect the **owner's personal** Discord/Reddit/Twitter/Instagram passwords and TOTP seeds via env/admin UI — Discord self-botting is a permanent-ban offense and a legal exposure. Recommendation: delete the subsystem and the credential collection.

HIGH orchestrate.ts:31–38; types.ts (both copies)

Attribution and drift problems corrupt the signal

Ticker attribution is substring-based — `YJ` (in the live default list) matches arbitrary text, so short tickers accumulate noise mentions. The #1 "top promoter" is literally `unknown@Reddit` because Serper mentions hardcode author "unknown". And there are **three** parallel scanner stacks (production lib, evaluation CLI, legacy real-social-scanner) with behaviorally drifted thresholds — the same ticker on the same day gets a different riskLevel depending on which path ran. Note also: **social results never feed back into any user-facing risk score** — they only annotate admin dashboards and the scheme database.

The fix (Phase 3): async + budgets + dedup + tiering

(1) POST returns 202 and execution moves out of the request handler with incremental writes — kills the #1 instability and the double-spend in one move. (2) Per-ticker streaming/chunking and per-request 10s timeouts. (3) Reddit via OAuth or drop it on serverless. (4) Unique constraint + cross-run seen-URL cache. (5) Serper `num:10` + merged query + top-20 tickers; Perplexity only for pre-flagged tickers. Estimated cost reduction: 60–80% per scan, with scans that actually complete.

5. Security review

Full detail: `findings/04-security.md`. 65 admin route files audited individually; injection/XSS/secret sweeps across the repo.

5.1 Critical & high

Severity	Finding	Impact / exploit
CRITICAL	Production DB credentials committed (<code>scripts/query-blogs-pg.mjs:6</code>)	Direct production Postgres access for anyone with repo read access. Rotate today (\$2).
HIGH	Quota bypass on <code>/api/ai-analyze</code>	

		Checks quota but never consumes it and is absent from the middleware matcher — FREE users get unlimited full ML analyses; the paid <code>/api/check</code> is trivially sidestepped.
HIGH	Apple receipt replay (<code>billing/apple/validate</code>)	No uniqueness check on <code>original_transaction_id</code> , no stored expiry, no downgrade: one purchased receipt shared = unlimited permanent PAID accounts.
HIGH	No rate limit on web login (NextAuth credentials)	Unthrottled password brute force against any account; mobile login is rate-limited, web is not; no CAPTCHA on this path.
HIGH	HTML injection into transactional email (<code>src/lib/email.ts</code>)	Contact-form name/subject/message interpolated raw into admin notification AND into a confirmation sent to an attacker-chosen address — a phishing primitive from your trusted sender domain.
HIGH	Preview-login OWNER backdoor (default password, shared prod DB)	See §2 E2.

5.2 Medium (summary)

- No security headers at all — no CSP, X-Frame-Options, HSTS, nosniff, Referrer-Policy (`next.config.js`); admin/account pages clickjackable.
- Register doesn't lowercase email while login does → MixedCase registrants can never log in; case-variant duplicate accounts.
- CAPTCHA optional on forgot-password (omit the token to skip it) → reset-email bombing.
- Mobile refresh tokens: stateless 7-day JWTs, no rotation/revocation, secret derived as `JWT_SECRET + "_REFRESH"`, algorithm not pinned.
- Password reset does not invalidate existing sessions; password policy inconsistent (10 vs 8 chars across flows).
- Upload route accepts SVG with client-declared MIME into a public bucket (stored-XSS vector); any admin role incl. VIEWER can upload.
- Admin blog preview renders stored HTML *without* DOMPurify (the public renderer sanitizes; the admin origin doesn't) — imported HTML files execute in the admin session.
- Credential-encryption key falls back to `NEXTAUTH_SECRET`; integration secrets are fanned out to GitHub Actions + Vercel preview envs; server-side storage writes use the public anon key (bucket RLS is the only barrier — and CI's `x-upsert` uploads prove anon *writes* are currently allowed).
- `/admin` pages are not in the middleware matcher — the entire admin UI shell is client-gated only (data APIs are protected; page shells are not).

5.3 Verified secure (checked, not vulnerable)

PayPal webhooks are signature-verified and fail closed; billing and user routes key off the session (no IDOR found); password-reset/verification tokens are 32-byte CSPRNG, single-use, atomically consumed; admin session tokens stored as SHA-256; inbound-email webhook uses timing-safe comparison and fails closed; Turnstile fails closed in production; login callbackUrl sanitized (no open redirect); no SQL injection (parameterized throughout; raw SQL is hardcoded DDL only); no `child_process` / `eval` ; admin mutations enforce role checks — 65/65 admin route files have auth on state-changing handlers.

6. Architecture & data layer

Full detail: `findings/05-architecture-data.md` (31 findings).

CRITICAL `package.json:8; prisma/` (no migrations dir)

`prisma db push --accept-data-loss` in the build path, zero migration history

Deploys diff-sync the schema straight into production and silently drop columns/tables on renames; failures are swallowed by `|| echo` ; preview builds can push their schema at the prod DB; there is no rollback or audit trail. Adopt `prisma migrate` with a discrete `migrate deploy` release step.

CRITICAL `src/lib/admin/scan-data.ts; scan-intelligence routes`

GitHub-as-database for the admin intelligence surface — unauthenticated, 60 req/hour

Every admin intelligence page reads pipeline JSON from the GitHub Contents API with no auth token: the history page alone issues 30 sequential calls; the stock deep-dive downloads up to ~30–40MB serially per request (no `maxDuration` → 504s); "pagination" re-downloads a 2MB file per page flip and "repairs" truncated JSON with `lastIndexOf("},")` , silently corrupting records. One admin with two tabs exhausts the hourly budget and all dashboards silently render empty. The fix is already half-built: the pipeline should upsert into the existing Postgres tables and admin routes should read Prisma.

CRITICAL `src/app/api/check/route.ts:282–299; src/lib/usage.ts:162–209`

Quota charged before validation, never refunded, and the reservation races

The scan slot is consumed before the request body is even parsed; every later failure (400/503/500) burns the user's quota with no result and no refund. The reserve itself is read-then-write — concurrent requests at the limit slip past it.

HIGH multiple

Serverless anti-patterns and observability gaps

Rate limiting fails open to per-instance memory Maps (auth tiers unprotected exactly when the DB hiccups); circuit breakers and caches are per-lambda; a module-level `setInterval` runs in lambdas; the un-awaited admin alert email races the response and is frequently lost at instance freeze. Sentry is wired but blind — only the global error boundary captures; all ~100 API routes swallow into `console.error` (443 call sites). `/api/check` chains ~8 serial stages including two with *unbounded* timeouts (OpenAI SDK default 600s; SEC fetch none) inside `maxDuration=30` — one slow call → 504 after quota burned. Nine GitHub workflows exist; **none runs tests/types/lint** — and the evaluation tree is excluded from tsconfig, so pipeline code importing prod modules is never type-checked.

HIGH prisma/schema.prisma; src/lib/admin/metrics.ts

Data-layer flaws

`ModelMetrics` uses read-modify-write counters on the hot path (concurrent scans corrupt the running averages permanently). `ApiUsageLog` grows forever with no index matching its main queries; `RateLimitEntry` rows are never deleted. Money is stored as `Float`; JSON lives in `String @db.Text`; several "foreign keys" are free-text with no relation (orphans). ~9 redundant indexes duplicate `@unique` constraints on the hottest tables while `ScanHistory(userId, createdAt)` is missing. The backfill route loads the entire ScanHistory table into lambda memory inside a 5s-budget transaction.

HIGH evaluation/; src/lib/social-scan vs evaluation copies; repo root

Duplication and dead weight

~70% of `evaluation/scripts` is superseded variants (8 `run-evaluation-*` + 3 social-scanner generations + dated shell scripts ≈ 5,300 dead/duplicate lines) feeding a 2,814-line single-file pipeline monolith. `platform-patterns.ts` is byte-identical in two trees while `types.ts` drifted behaviorally. The repo root carries a case-collision (`plan.md` + `PLAN.md` — breaks macOS/Windows checkouts), an accidental `Untitled` file, duplicate scripts, a dead embedded subproject (`scamdunk-history-db/` with a second Prisma schema), and 8.8MB of evaluation data shipped with every deploy. The README documents Stripe; the app uses PayPal.

What breaks first at 10× traffic: (1) GitHub 60 req/h — admin dashboards go dark in minutes; (2) Postgres connections (per-request rate-limit writes + interactive transactions + 18-query dashboard bursts); (3) the `/api/check` 30s budget; (4) seq scans on the unpruned event tables; (5) quota race overage.

7. Frontend & admin UI

Full detail: `findings/06-admin-ui.md` (36), `findings/07-admin-components-editor.md` (22), `findings/08-public-frontend.md` (37).

7.1 Product-integrity bugs (public)

Severity	Finding	Impact
HIGH	Uploaded screenshots silently discarded (<code>ScanInput.tsx:147</code> → <code>HomeContent</code> → API)	The UI toasts "Screenshots will be analyzed for scam patterns" but images never reach the backend. On a fraud-detection product, telling users evidence was analyzed when it wasn't is the single worst UX finding in this audit.
HIGH	Email-verification prompt is dead code (<code>login/page.tsx:53</code>)	NextAuth v5 returns the error code in <code>result.code</code> , not <code>result.error</code> — unverified users always see "Invalid email or password" and never get the resend option. Signup-funnel leak; one-line fix.
HIGH	Sidebar scan history items do nothing when clicked	Tapping a past scan just closes the panel — no detail route, no callback.
MEDIUM	Scan progress is fully simulated (~6.2s of <code>setTimeouts</code>)	Stamps "SEC and alert databases checked" regardless of outcome; keeps animating behind the error screen on failure.
MEDIUM	LimitReached shows PAID users "free checks" copy + a subscribe button for the plan they already pay for	Duplicate-subscription risk; also illegible in dark mode.
MEDIUM		

	Turnstile widget destroyed/recreated on every keystroke; PayPal button re-fetches config + tears down on every parent re-render	CAPTCHA churn can drop a completed verification; checkout button can reset mid-flow.
MEDIUM	Admin invite auto-login posts an empty email — always fails silently	Every invited admin lands on a blank login form with no explanation.
MEDIUM	SEO: <code>/og-image.png</code> referenced but doesn't exist; SearchAction advertises a search that doesn't exist; ticker validation rejects BRK.B/1INCH; author bylines 404 outside a 2-entry hardcoded map	Broken share cards, structured-data risk, blocked legitimate scans, crawlable 404s from every article.

7.2 Systemic admin-UI defects

- **The shared hook exists but is dead code.** `useAdminFetch` has zero importers; ~21 pages hand-roll the same fetch/loading/error block with three recurring defect classes: **no AbortController anywhere** (stale responses overwrite newer data on every filterable page — users, support, social-scan, stock-lookup, email-management, api-usage, model-efficacy...), **stuck-error pages** (market-analysis, risk-alerts, scheme/promoter/stock detail pages never clear `error` and gate the whole page on it — one transient failure bricks the page until remount), and **no 401 → login redirect** after session expiry.
- **Silent failures across the news CMS** — failed deletes/publish toggles give zero feedback; failed list fetches render as "No posts found".
- **Destructive actions without confirmation** — browser-agents "Clear 30d+", api-usage alert delete, user plan downgrades; plus double-submit bugs (model-efficacy feedback double-clicks skew the accuracy metrics the team uses to tune the model).
- **TipTap editor** (verified against installed v3.20): missing `immediatelyRender:false` (throws in dev), stale toolbar state (v3 no longer re-renders on transaction), duplicate Link/Underline extensions defeating `openOnClick:false`, full-document serialization per keystroke, ~700KB mammoth in the page bundle, base64 image paste ballooning post content, and the admin preview rendering imported HTML *unsanitized* (XSS in the admin origin — the public side sanitizes correctly).
- **Render-phase state mutation** (promoter + data-ingestion pages call `.sort()` directly on React state), keyless fragments on expandable tables, and an a11y debt across the board (unlabeled inputs, icon-only buttons, color-only risk indicators, hover-only disclosure panels, modals without dialog semantics).

8. Automated check results

Check	Result
-------	--------

<code>npm audit</code>	18 vulnerabilities: 1 critical (handlebars — prototype pollution/JS injection; transitive), 9 high (undici ×6 advisories, xmldom ×5, glob), 8 moderate (uuid, ws, brace-expansion...). <code>npm audit fix</code> covers most without breaking changes.
<code>tsc --noEmit</code>	10 type errors , all in <code>src/tests/e2e.test.ts</code> (optional vs required behavioral-context fields). The suite passes only because ts-jest is configured leniently; CI would catch this — there is no CI.
<code>jest</code>	58/58 tests pass in 4 suites — but a worker leaks (force-exited; untracked timer), <code>e2e.test.ts</code> is misnamed unit tests over mocks, and <code>rate-limit.test.ts</code> asserts only static config constants. None of the bug classes found in this audit (quota races, route auth, scan-data parsing) is covered.
Python deps	Known-vulnerable pins: gunicorn 21.0 (CVE-2024-1135), requests 2.31 (CVE-2024-35195), fastapi 0.109/python-multipart (CVE-2024-24762), tensorflow 2.15 (multiple); yfinance 0.2.30 is ~2.5 years stale and the proximate cause of live 503s.
Secrets scan	One live credential committed (§2 E1). <code>.env</code> correctly gitignored otherwise; <code>.env.example</code> files are placeholders.

9. Findings index by area

Every finding with file:line evidence, exploit/impact detail, and a concrete fix lives in the appendix files. Counts below; IDs are referenced throughout the roadmap. Some findings are independently confirmed by multiple reviewers (e.g. the blog-preview XSS, the JsonLd escaping gap, the SEC substring check) — they are counted in each area's file but fixed once.

Area	Appendix	Critical	High	Medium	Low	Total
AI scoring engine (TypeScript)	<code>findings/01-ai-scoring-typescript.md</code>	7	9	14	9	39
Python AI/ML service	<code>findings/02-python-ai-service.md</code>	6	9	13	7	35
Social scanning subsystem	<code>findings/03-social-scan.md</code>	3	13	16	14	46
Security	<code>findings/04-security.md</code>	1	5	10	10	26
Architecture & data layer	<code>findings/05-architecture-data.md</code>	4	9	13	5	31
Admin UI pages	<code>findings/06-admin-ui.md</code>	0	11	22	3	36
Editor & shared admin components	<code>findings/07-admin-components-editor.md</code>	0	5	11	6	22
Public frontend, auth, SEO	<code>findings/08-public-frontend.md</code>	0	4	16	17	37
Total		21	65	115	71	272

10. What is working correctly

Credit where due — these were explicitly verified, not assumed:

- PayPal webhook signature verification (fails closed); billing ownership checks via `custom_id`; no IDOR across user routes.
- Token hygiene: CSPRNG reset/verification tokens, atomic single-use consumption, hashed admin session tokens, expiring invites.
- All 65 admin API route files enforce auth on state-changing handlers; role checks on mutations.
- Turnstile fails closed in production; callbackUrl sanitized; no SQL injection; no `eval` / `child_process` in app code.
- Public blog rendering sanitizes with DOMPurify; TipTap v3's link handling blocks `javascript:` URIs (verified against installed sources).
- The scoring test suite (525 lines) is genuinely useful; the data-ingestion admin page has the most robust fetch handling in the codebase; the public Sidebar demonstrates the correct AbortController pattern the admin pages should adopt.
- Prisma client singleton and scan-status interval cleanup are implemented correctly.

11. Remediation roadmap (summary)

The full dependency-ordered plan with effort estimates and finding-ID traceability is in `ROADMAP.md`. Shape:

Phase	Window	Theme	Headline items
0 — Emergency	Today (~2h)	Stop active exposure	Rotate DB password · delete preview OWNER · cancel Apify · enforce AI_API_SECRET
1 — Stop the bleeding	Week 1	Billing/auth/ deploy safety	Meter ai-analyze · Apple receipt binding · quota validate-then-reserve+refund · login rate limit + Turnstile · email escaping · security headers · prisma migrate (kill <code>db push --accept-data-loss</code>) · CI workflow · npm/pip audit fixes
2 — Trustworthy scoring	Weeks 2–3	The "AI not working" fix	One shared scorer (delete the eval fork) · signal de-dup + recalibration · unknown≠0 + confidence field · SEC layer rebuild · RF feature contract + no in-request retraining · OTC mapping · remove synthetic fundamentals · zod on the backend contract · LLM timeouts/caching · coherence invariants
3 — Social scans	Weeks 3–4	Stable + 60–80% cheaper	Async execution + incremental writes · scanner time budgets + per-fetch timeouts · Reddit OAuth · unique-key dedup + seen-URL cache · Serper/ Perplexity tiering · word-boundary attribution · delete browser agents · lock storage buckets
4 — Data layer & admin	Weeks 4–5	Postgres as source of truth	Pipeline → Postgres, delete GitHub-API data layer · KV rate limiting (fail closed) · <code>withApiHandler</code> + Sentry on all routes · atomic metrics · schema migrations (Decimal/Json/FKs/indexes) · retention cron · upload + token hardening

5 — Frontend & hygiene	Week 6	UX correctness	useAdminFetch (abort/401/error-clear) across ~21 pages · sanitize admin preview · TipTap fixes · screenshot feature wired or removed · email-verified login fix · confirmations + disabled states · repo cleanup (case-collision, dead subprojects, 11 dead pipeline variants)
6 — Calibration & quality	Ongoing	Keep it working	Recalibrate unified scorer on labeled data · integration tests + strict TS · real ML training data or keep ML off · cost/observability dashboards

12. Methodology & appendices

Seven specialized adversarial reviews ran in parallel, each reading its assigned files completely (line-by-line, with cross-file contract verification — e.g. the TS ↔ Python API contract was checked from both sides; TipTap behaviors were verified against the installed v3.20 sources in `node_modules`; the RF feature mismatch was reproduced empirically). Their outputs were cross-checked against each other and against automated tooling (`npm audit` , `tsc` , `jest` , dependency review, secret grep, git history analysis). Findings independently confirmed by two or more reviewers are flagged in the appendix files.

- `audit/2026-06-11/ROADMAP.md` — full remediation plan (phases, effort, finding-ID traceability)
- `audit/2026-06-11/findings/01..08-*.md` — every finding with evidence, impact, and concrete fix

Prior internal audits (`SECURITY_SCALABILITY_AUDIT.md` , `RED-TEAM-AUDIT-REPORT.md`) were reviewed for context: several of their critical items have since been fixed (PayPal webhook verification, CAPTCHA fail-closed, admin init/setup auth) and are acknowledged in §10; items still open (in-memory rate limiting, preview-login, refresh-token secrets, Python API hardening) are re-confirmed in this report with current line numbers.

ScamDunk full-codebase audit · 2026-06-11 · Generated from branch `main` @ 30976b1 · Deliverables: this report (HTML/PDF), ROADMAP.md, findings appendices.